环境与向量执行过程

西安电子科技大学　香山芯锐

建立可运行、可观察的整核实验环境，追踪同一条 VADD 从取指到已提交映射的过程。

赛题 1.1、1.2、1.3 | 实验材料截至 2026-10-30 | 第五版 · 2026-11-04

1 项目概述与任务对应

本项目为香山昆明湖 V2 增加了 `vdot.vv` 指令。它从两个向量寄存器中各取八个有符号 8 位数，逐项相乘，再把八个乘积加起来。处理器用一个微操作完成这次点积，结果通过原有的向量写回和退休通路提交。NEMU 参考模型也加入了相同的指令规则，用于逐次核对硬件运行结果。

边缘 LLM 的 INT8 线性层反复计算输入与权重的点积。本项目把八项点积放入流水线中的专用功能单元，沿用向量寄存器、重命名、发射、写回和退休机制，再用软件累加多个部分和，完成较长的点积。

报告按工程实施内容分为三个阶段。第一阶段说明环境部署、程序运行及 VADD 执行过程；第二阶段给出指令规格、硬件结构、RTL 接入和功能测试；第三阶段说明参考模型、差分检查与性能测量。源码、原始日志和波形材料通过各阶段附录定位。

1.1 计算目标与工程方案

设两个源寄存器的低八个字节分别为有符号整数 $a_0 \ldots a_7$ 和 $b_0 \ldots b_7$，无掩码操作计算：

$$S = a_0b_0 + a_1b_1 + \ldots + a_7b_7$$

八个乘积从零开始求和，不把目的寄存器的旧值加进去。硬件先用八路乘法器同时计算，再用三级加法树求和；第一拍保存乘积，第二拍保存整理好的结果。输入怎样从内存装入、结果何时存回、多次点积能否共用输入，都由程序安排。因此，性能测试既测寄存器已经备好数据时的计算，也测包含加载和存储的完整任务。

设计以赛题规定的固定八项 INT8、归约初值为零和其余输出清零为基础。项目进一步定义编码、掩码、有效长度、寄存器重叠和状态更新规则，形成 RTL、NEMU 与测试共同采用的指令规格，具体定义见第 4 章。

1.2 主要技术结果

在指定处理器配置下，基础算术、异常、寄存器重叠和流水场景均已完成整核差分检查。标准 RVV 等价实验组合了 144 组输入配置，三条实现共执行 432 次内核调用。波形分析另对正常执行、投机取消、退休状态请求及发射前资源竞争进行了动态指令追踪。

让后一次点积读取前一次的结果，连续执行时，每增加一次操作，VDot 需要增加 7 周期，RVV-m1 和 RVV-mf2 分别增加 55、47 周期。两条 RVV 路径与 VDot 的延迟比分别为 7.86 和 6.71。统计包含各实现为得到相同结果而执行的配置、计算和结果整理。单独看 VDot 功能单元，从接受输入到完成计算为 2 周期；这段时间在第 6 章的波形中测量。

把输入加载和结果存储也算进去后，初版程序执行 128 次点积，VDot/RVV-mf2 用 2239/2241 周期，耗时基本相同。双方都优化加载安排后，G=16 分段任务的周期比为 1.24。双方都复用输入时，2×2、K=256 内核用 790/1043 周期，周期比为 1.32，VDot 耗时少 24.3%。第 9 章分析这些结果与 INT8 线性层的联系；完整模型的生成速度还需计入量化处理、矩阵遍历及运行时开销。

1.3 赛题要求与报告内容

表 1-1 按项目保存的赛题要求归档组织技术材料。每篇正文承担明确职责，源码和日志通过各阶段附录定位。

表 1-1　三阶段技术要求与阅读位置。

赛题项目	需要说明的内容	本套正文
1.1、1.2	定制 Hello、环境部署和操作记录	第 2 章：环境部署与运行验证
1.3	基于波形的向量加法执行过程	第 3 章：VADD 指令执行过程
2.1	指令语义、微架构和数据通路	第 4 章：指令规格与微架构设计
2.2	功能单元 RTL 与整核扩展	第 5 章：RTL 实现与整核接入
2.3.1～2.3.3	正数、负数和边界值功能测试	第 6 章：功能测试与补充时序验证
3.1.1、3.1.2	NEMU 扩展及 RTL/NEMU 协同仿真	第 7 章：NEMU 扩展与差分验证
3.2	标准 RVV 指令组合的等价实现	第 8 章：标准 RVV 等价基线
3.3	Baseline 与 VDot 的延迟及加速比	第 9 章：延迟测量与软件优化分析

第 4～7 章说明硬件设计、整核接入及验证过程，第 9 章分析微架构执行成本与软件组织的关系。评分权重和原文出处见项目需求矩阵。

1.4 配套材料与阅读方式

本套报告包含分章正文、三阶段合订稿和复现索引。正文保留主要代码、测试方法与数据分析，附录列出版本、校验值和完整材料路径。作品仓库、演示视频及提交状态由阅读导航提供入口。

本套沿用项目保存的官方要求及截至 2026-10-30 的实验材料，官方条文对照见2026-10-29 提交要求归档。最终上传时按平台实际接受的格式、大小和位置选择对应文件。

2 环境部署与运行验证

先确认程序能在香山仿真器中启动、输出并正常退出，再确认向量运算能给出正确结果。为此，本阶段运行定制 Hello 和四元素 VADD：Hello 打印指定文字，VADD 把 `[1,2,3,4]` 与 `[10,20,30,40]` 相加，得到 `[11,22,33,44]`。第 3 章再用保存的波形查看处理器如何完成这条加法指令。

2.1 实验平台与仿真流程

基础实验在 Windows 11 的 WSL2 Ubuntu 中完成，处理器采用昆明湖 V2 代码线的单核 MinimalConfig。表 2-1 列出 2026-10-19 环境验收时的配置；第三阶段支持自定义指令的源码 NEMU 在相应附录单独登记。

表 2-1　基础环境配置。

项目	本次实验配置
Linux 环境	WSL2，Ubuntu 24.04.4 LTS
可用资源	主机约 16 GB 内存、20 个逻辑处理器；WSL 内存约 9.7 GiB、swap 8 GiB
香山源码	16ae9ddc，单核 MinimalConfig，VLEN=128
波形支持	构建时启用 `EMU_TRACE=1`
Java / Mill	OpenJDK 21.0.12.1 / 工程锁定 Mill 0.12.3
RTL 仿真	Verilator 5.052 生成的 `build/emu`
应用工具链	RISC-V GCC 13.3.0、binutils 2.42、Make 4.3
AM 环境	nexus-am 086a2380
第一阶段参考模型	ready-to-run 仓库 ed8c30e1 的预编译 NEMU 共享库

编译后要得到三个配合工作的文件。香山 Chisel 源码先生成 RTL，再由 Verilator 转成可以运行的仿真器 `emu`；测试程序经 RISC-V 编译器生成镜像；NEMU 提供参考执行结果。运行时，emu 读取测试镜像，模拟处理器执行，并把寄存器等架构状态交给 DiffTest，与 NEMU 比较。只改测试输入或打印文字时，重新编译应用即可；改了 RTL 才需要重建 emu。

表 2-1 和本章原始截图记录的是第一阶段的 WSL 实验。当前提交仓库已经提供 PowerShell 调用 Docker 的复现入口，见下节。两套记录分别保存版本和文件校验值：早期实验使用预编译 NEMU，当前 VDot 复现使用包含自定义指令的源码 NEMU。

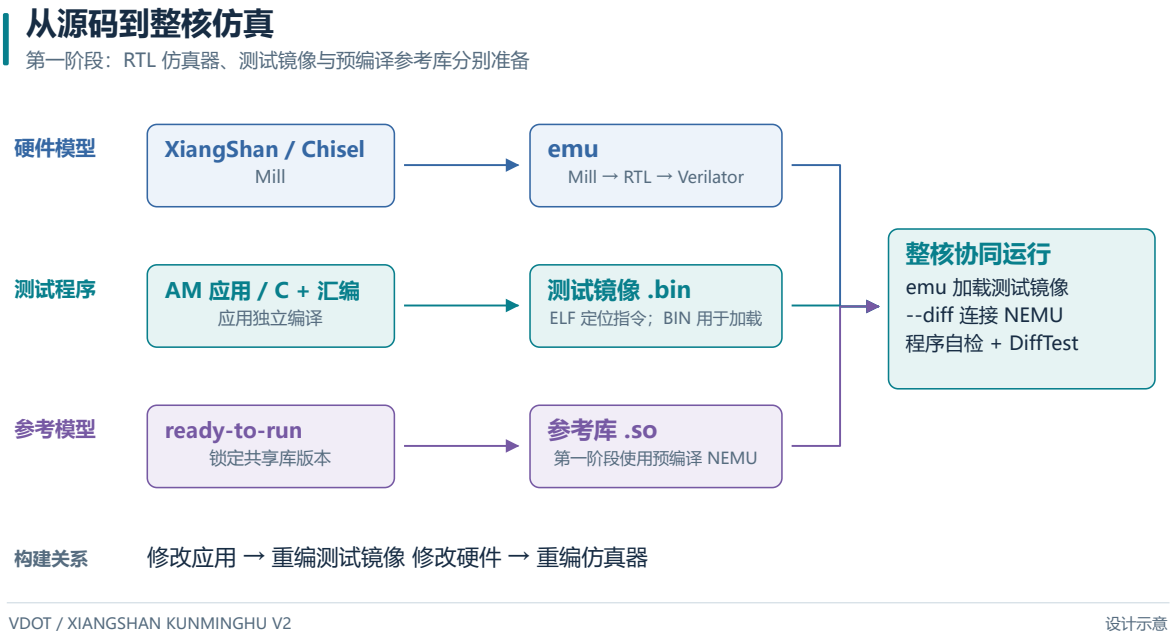


图 2-1　第一阶段的编译与运行关系。硬件生成 emu，应用生成测试镜像，运行时连接 ready-to-run 中锁定的预编译参考库。

2.2 当前复现入口与原始构建记录

2.2.1 在提交仓库中运行 Hello 和 VADD

本机已完成 Docker 环境和整核编译。先启动 Docker Desktop，在 Windows PowerShell 中进入提交仓库，再逐条运行应用：

```
Set-Location 'D:\Mayyyyyy\WorkSpaces\日常工作区1\2026比赛规划
\RISCV_vdot\submission_ready\cie2026-xiangshanxinrui-vdot'
python tools/container.py exec python3 tools/recording_case.py hello
python tools/container.py exec python3 tools/recording_case.py vadd4
```

运行脚本会先检查所需文件，再执行仿真。终端打印的 `Run directory` 保存本次日志、命令和退出状态。运行后依次核对应用输出、`Difftest enabled.`、`HIT GOOD TRAP`、退出码 0 和 `RUN_CHECK=PASS`。新机器从提交仓库的 REPRODUCE 完成首次准备。

2.2.2 第一阶段的 WSL 构建记录

第一阶段先准备 Java、RISC-V 交叉编译器、LLVM/Clang 和 Verilator，再载入 xs-env 环境。恢复构建时曾报 `clang++: No such file or directory`，退出码为 127。原因是 shell 的 PATH 中没有编译器所在的 `/usr/local/llvm/bin`。补上该目录后，RTL 生成、Verilator 转换和 C++ 编译继续完成。

以下是归档记录使用的 WSL/Bash 构建路径和命令：

```
cd /mnt/d/Mayyyyyy/WorkSpaces/VScode/xs-env/XiangShan
export PATH="/usr/local/llvm/bin:$PATH"
make emu CONFIG=MinimalConfig EMU_TRACE=1 JVM_XMX=6G -j4
```

`CONFIG` 选择处理器配置，`EMU_TRACE` 保留导出波形所需支持，`JVM_XMX` 和 `-j4` 分别控制 JVM 堆和编译并行度。编译完成后得到该配置下的仿真器 `build/emu`。构建使用已有工程的恢复方式，原错误和各阶段退出状态均保留在部署记录〔1〕及构建时间记录〔2〕中。

2.3 Hello 程序与应用加载

Hello 调用 AM 的打印接口，输出成员标识和当时的 WSL IPv4，然后返回。源码中的 `172.26.232.122` 是首次实验记录的固定地址；重新运行时会原样打印，不会查询当前电脑的 IP。对应语句如下：

```
printf("hello xiangshan, I am Meijie, IP address: 172.26.232.122\n");
```

先在 WSL/Bash 编译 AM 应用：

```
export AM_HOME=/home/lenovo1227/xiangshan-learning/nexus-am
cd "$AM_HOME/apps/hello-meijie"
make ARCH=riscv64-xs LINUX_GNU_TOOLCHAIN=1 MARCH=rv64gcv -j2
```

`ARCH=riscv64-xs` 选择香山运行环境，`LINUX_GNU_TOOLCHAIN=1` 选择对应工具链前缀，`MARCH=rv64gcv` 指定应用指令集。编译得到的 `.bin` 用作仿真器输入，ELF 与反汇编用于定位符号和指令。

下列命令加载本次保存的镜像，显式指定差分参考库：

```
XS_ENV=/mnt/d/Mayyyyyy/WorkSpaces/VScode/xs-env
RUN_DIR='/mnt/d/Mayyyyyy/WorkSpaces/日常工作区1/2026比赛规划/RISCV_vdot/artifacts/runs/S1-
01/hello-meijie-20261019-120654-8FtdkQ'
cd "$XS_ENV/XiangShan"
./build/emu \
  -i "$RUN_DIR/hello-meijie-riscv64-xs.bin" \
  --diff "$XS_ENV/XiangShan/ready-to-run/riscv64-nemu-interpreter-so" \
  --no-snapshot -C 100000 -s 1
```

`-i` 告诉 emu 运行哪个应用镜像，`--diff` 告诉它用哪个 NEMU 库比较结果。`-C` 限制最多模拟多少周期，`-s` 固定随机种子，`--no-snapshot` 关闭快照。成功运行应同时出现预期文字、DiffTest 启用和 GOOD TRAP，程序退出码为 0。

2.4 运行结果及向量环境检查

归档日志和独立退出记录给出以下结果：

```
BUILD_EXIT_CODE=0
Difftest enabled.
hello xiangshan, I am Meijie, IP address: 172.26.232.122
HIT GOOD TRAP at pc = 0x8000014c
EMU_EXIT_CODE=0
```

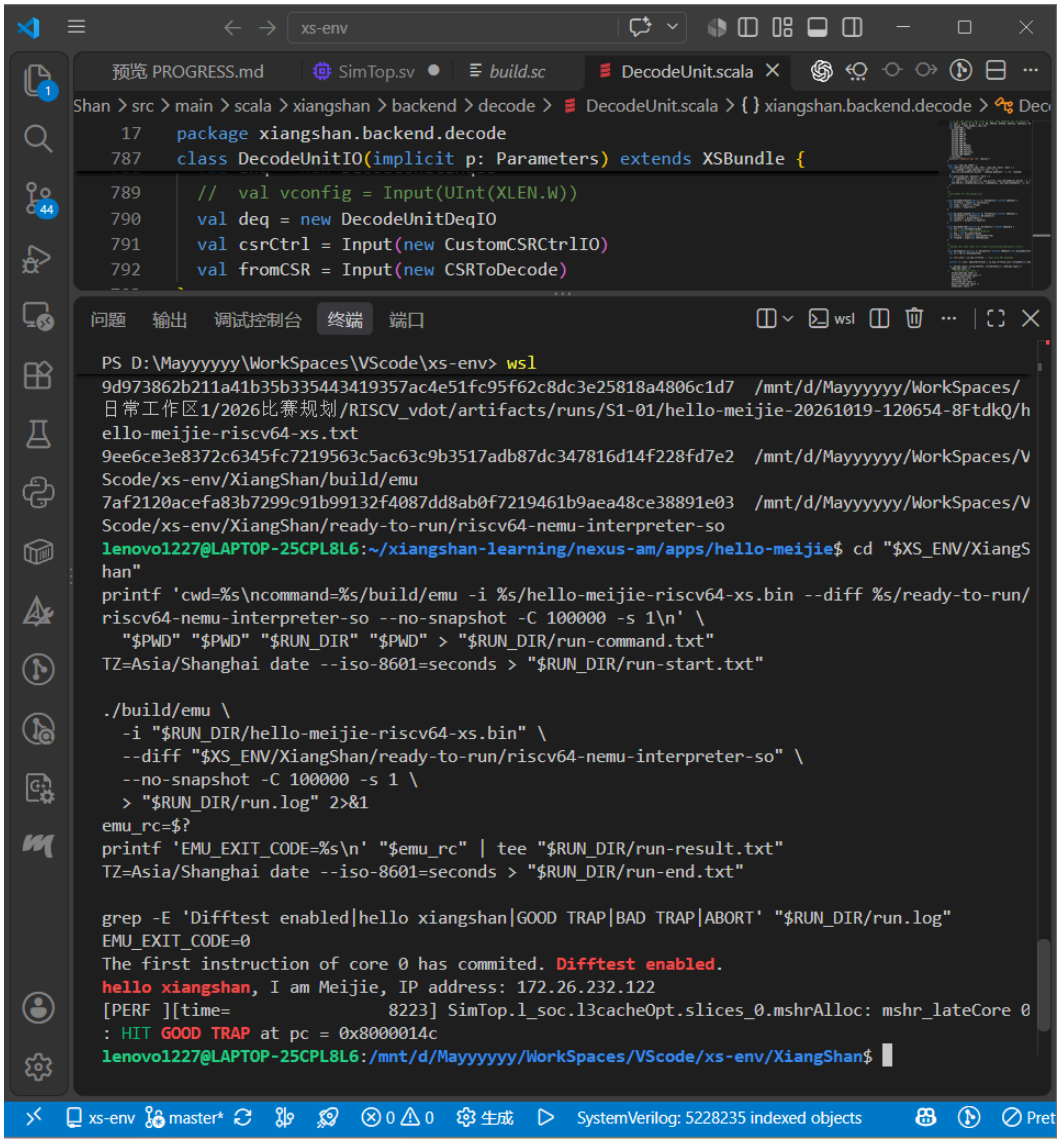


图 2-2　定制 Hello 的运行结果。个人标识、当次 IP、GOOD TRAP 与退出状态共同确认程序完成。

四元素 VADD 以 `[1,2,3,4]`、`[10,20,30,40]` 为输入，应用逐项检查结果 `[11,22,33,44]`。首次运行和录波运行均启用 DiffTest，并正常结束。至此，基础环境已经完成程序启动与输出、向量装载、运算和存储的检查。

下一章沿同一条 VADD 的动态执行实例，说明取指到提交各阶段如何衔接。相应的 Hello 原始日志〔3〕、退出记录〔4〕和环境核对记录〔5〕列入附录，供复现时核查。

3 VADD 指令执行过程

本章跟踪一条 `vadd.vv v3,v1,v2`：v1 中是 `[1,2,3,4]`，v2 中是 `[10,20,30,40]`，相加后写入 v3。目标指令位于 PC `0x80000148`，使用 e32/m1、VL=4。沿波形依次查看指令被取回、译码、等待操作数、完成加法、写入物理寄存器，最后按程序次序退休。每一步都核对指令编号，避免把前后两次执行拼在一起。

3.1 测试程序和波形来源

测试配置为 VLEN=128、SEW=32、LMUL=1、VL=4，无掩码执行。数组 A=`[1,2,3,4]`、B=`[10,20,30,40]` 分别进入 v1、v2，结果存回 C。核心汇编如下：

```
li t0, 0x200
csrs mstatus, t0
csrwi vstart, 0
vsetvli t1, a3, e32, m1, ta, ma   # a3=4，检查实际VL
bne t1, a3, .Ldone
vle32.v v1, (a0)
vle32.v v2, (a1)
vadd4_target:
vadd.vv v3, v1, v2
vse32.v v3, (a2)
.Ldone:
mv a0, t1
ret
```

C 程序将输出预填为 −1，并逐项比较独立期望。运行读回 `vlenb=16`，确认 128 位向量宽度，随后给出四项结果和正常结束标志：

```
vadd4: VL=4 SEW=32 LMUL=1 VLEN=128 bits
C[0] = 11 (expected 11)
C[1] = 22 (expected 22)
C[2] = 33 (expected 33)
C[3] = 44 (expected 44)
PASS: vadd.vv produced [11, 22, 33, 44]
Core 0: HIT GOOD TRAP at pc = 0x8000023c
```

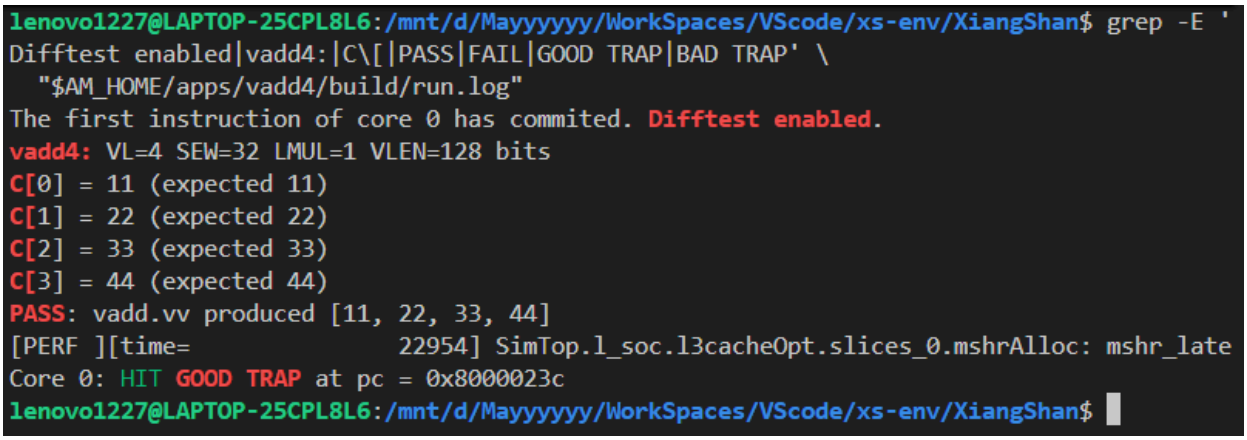


图 3-1　VADD 应用结果。结束陷阱位于 `0x8000023c`，本章追踪的加法指令位于 `0x80000148`。

本章截图和时间点对应第一阶段保存的 VCD。复查时，在提交仓库根目录用 `python tools/prepare_recording.py --only vadd4` 还原波形，再用 `python tools/open_wave.py 101-fetch` 打开取指视图；完整视图与工具说明见波形复查入口。下面保留生成原波形时的 WSL/Bash 命令，便于追溯应用怎样编译、波形怎样导出：

```
export AM_HOME=/home/lenovo1227/xiangshan-learning/nexus-am
XS_ENV=/mnt/d/Mayyyyyy/WorkSpaces/VScode/xs-env
cd "$AM_HOME/apps/vadd4"
make ARCH=riscv64-xs LINUX_GNU_TOOLCHAIN=1 MARCH=rv64gcv -j2

mkdir -p "$HOME/xiangshan-learning/waves"
wave_dir=$(mktemp -d "$HOME/xiangshan-learning/waves/vadd4-XXXXXX")
cd "$XS_ENV/XiangShan"
./build/emu \
  -i "$AM_HOME/apps/vadd4/build/vadd4-riscv64-xs.bin" \
  --diff "$XS_ENV/XiangShan/ready-to-run/riscv64-nemu-interpreter-so" \
  --no-snapshot --dump-wave-full \
  --wave-path="$wave_dir/vadd4.vcd" \
  -b 0 -e 8000 -C 200000 -s 0 \
  > "$wave_dir/run.log" 2>&1
wave_rc=$?
printf 'EMU_EXIT_CODE=%s\n' "$wave_rc" | tee "$wave_dir/result.txt"
```

`--dump-wave-full` 导出完整层级，`-b/-e` 选择仿真 cycle 窗口，`-C` 限制总周期，`-s 0` 与归档运行的种子一致。首次运行和导波形运行均通过 DiffTest，退出码为 0。复现时可利用生成的 ELF/反汇编定位符号，再在 GTKWave 中载入 VCD；操作入口见原波形指南〔6〕。

3.2 动态指令标识与时序总览

先用 PC=`0x80000148` 和机器码=`0x021101d7` 找到程序中的目标指令。前端用 FTQ `0/3` 标记它所属的取指块，目标位于第 `0xA` 个半字。进入后端后，这次执行分到 ROB `1/16` 和物理目的 51；逻辑源 v2、v1 则从物理寄存器 50、48 取值。后续波形都沿这组编号查找。

本次 VCD 每 2 ps 一个周期，偶数 ps 是上升沿。对于 valid/ready 接口，只有上升沿到来前两者都为 1，数据才在该沿被接受，所以 Marker 通常放在沿前 1 ps。例如，判断 5080 ps 接受了哪个请求，要看 5079 ps 的编号；5080 ps 沿后的接口可能已经换成下一项。退休时同时查看 `isCommit`、`commitValid` 和该槽的指令编号。ps 是本次仿真的时间标尺。

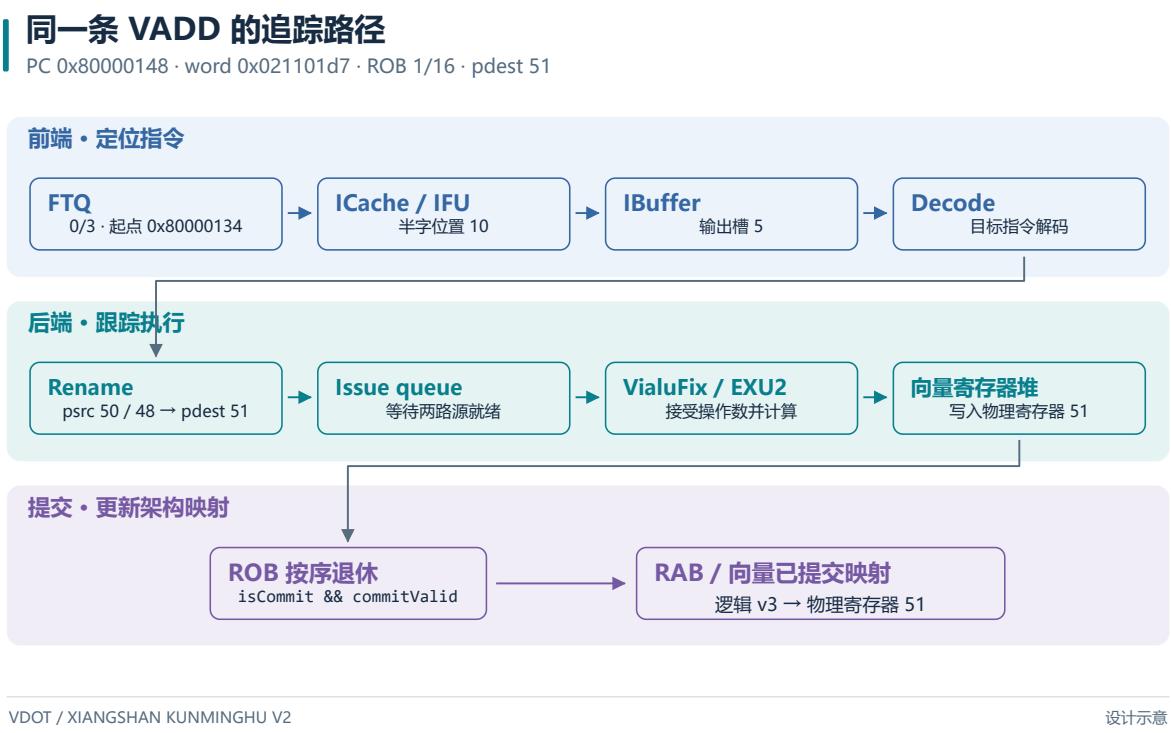


图 3-2　分析路径。前端追踪目标指令所属的取指块，后端追踪操作数就绪、执行、写回和提交过程。

表 3-1 以 FTQ 请求接受沿为相对周期零点，汇总本次动态执行的主要事件。后续各节结合有效握手和指令标识说明具体过程。

表 3-1　目标动态指令的主要事件。

事件	VCD 时间	相对周期
FTQ 取指请求接受	5080 ps	0
ICache 应答与 IFU F2 推进	5084 ps	2
IFU 输出进入 IBuffer	5086 ps	3
后端入口接受，目标进入 decodeBuf	5088 ps	4
译码接受目标	5112 ps	16
重命名输出接受	5116 ps	18
派发入队	5118 ps	19
两个物理源先后就绪	5252、5258 ps	86、89
发射接口接受	5262 ps	91
执行单元接受操作数	5268 ps	94
运算输出握手	5270 ps	95
物理寄存器写入	5272 ps	96
ROB 退休	5274 ps	97
v3 已提交映射更新	5278 ps	99

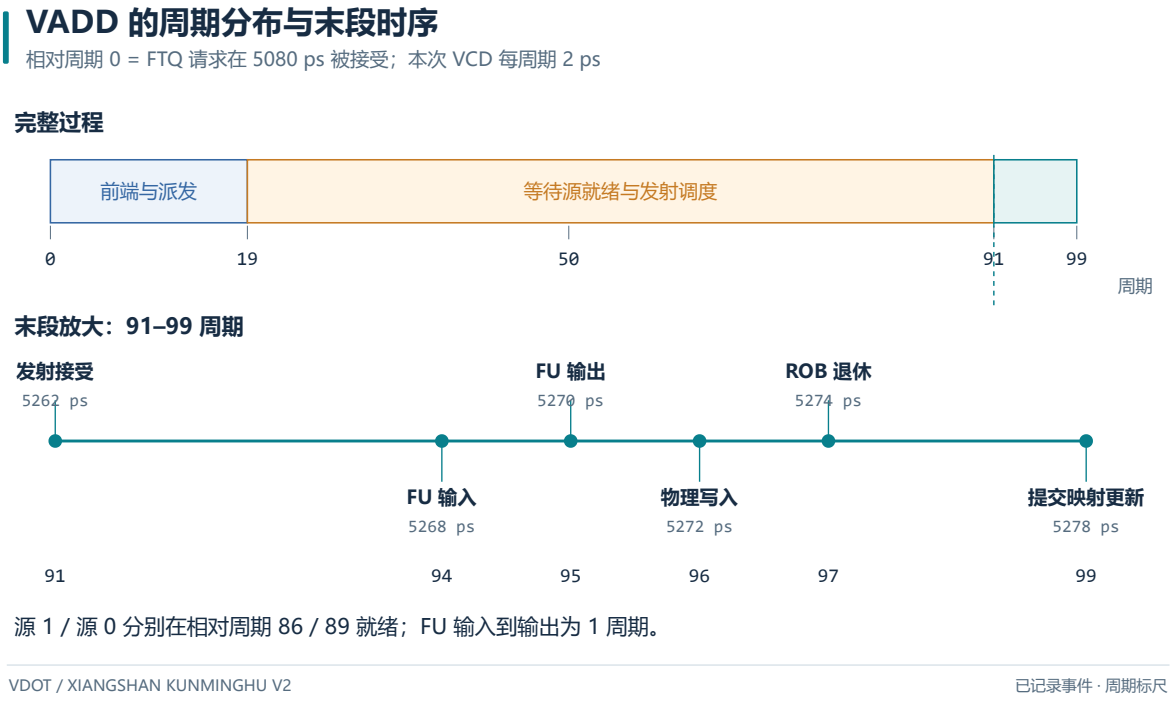


图 3-3　VADD 的周期分布。上图按完整 99 周期绘制，下图放大 91～99 周期；事件时间来自表 3-1。

阶段一附录列出完整信号层级及对应源码位置，正文中的信号名称按观察对象作适当缩写。

3.3 前端取指与译码交接

3.3.1 FTQ 请求与 ICache 返回数据

5079 ps 时，FTQ 请求的 valid/ready 均为 1，身份为 `0/3`，起始地址为 `0x80000134`。请求在 5080 ps 被接受，沿后接口立即转向 FTQ `0/4`、地址 `0x80000154`。这说明接口能够连续发送取指块；目标指令位于前一块内部，所以它的 PC 可以不同于该块的起始地址。

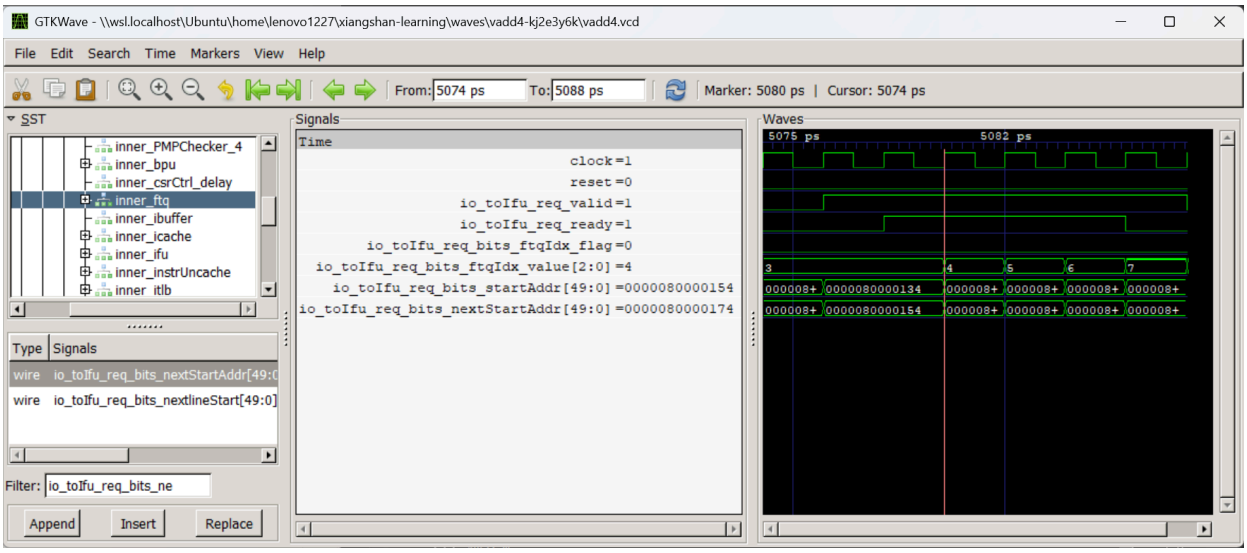


图 3-4　5080 ps 接受 FTQ `0/3` 后，接口转向下一取指块。目标所属请求应读取接受沿之前的 FTQ 编号。

5083 ps，ICache 返回有效应答，地址仍为 `0x80000134`；IFU F2 同时持有 FTQ `0/3`，`f2_fire=1`。本次请求跨越两条缓存行，`doubleline=1`，两侧异常字段均为零。ICache 应答为 Valid 接口，结合对应地址和 IFU 推进条件确认 5084 ps 的交接。

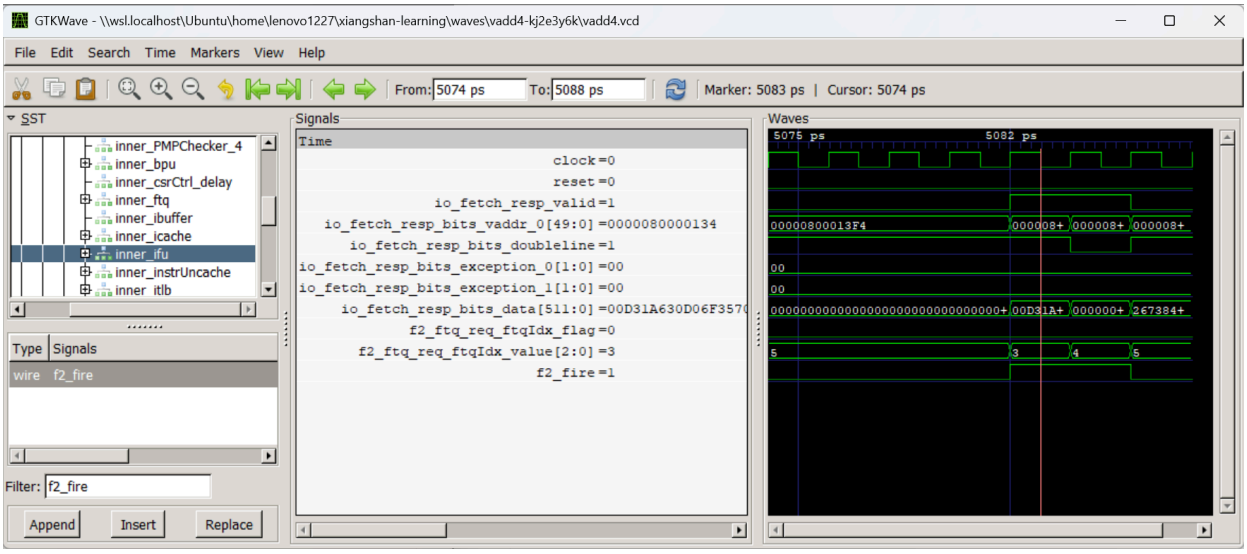


图 3-5　ICache 的有效应答与 IFU F2 中的 FTQ `0/3` 对应。

返回总线最低 128 位为 `020661A7_021101D7_0205E107_02056087`，右端是低位。目标机器码位于 `[95:64]`：

表 3-2　返回数据中的四条连续指令。

位段	地址	机器码与操作
[31:0]	0x80000140	0x02056087，装载 v1
[63:32]	0x80000144	0x0205e107，装载 v2
[95:64]	0x80000148	0x021101d7，目标 VADD
[127:96]	0x8000014c	0x020661a7，存储 v3

请求地址从前一条缓存行的末尾开始，但返回总线按选中的 8 字节 bank 重新拼接，最低部分对应 `0x80000140`。IFU 再按半字切分指针取出指令：目标使用 `f2_cut_ptr_10/11=36/37`，对应原数据的半字 4/5。因此，查目标机器码在返回总线的哪一段，要顺着切分指针看；只用 PC 减请求起点，会算到错误位置。

3.3.2 IFU 输出和 IBuffer 接收

目标距取指起点 20 字节，即第 10 个半字位置。5085 ps，IFU 位置 10 的 PC、机器码分别为 `0x80000148`、`0x021101d7`，包内 `valid` 和 `enqEnable` 均为 `0xd555`，其 bit10 有效；整包 valid/ready 为 1/1，接收侧没有 flush。目标由此在 5086 ps 进入 IBuffer。

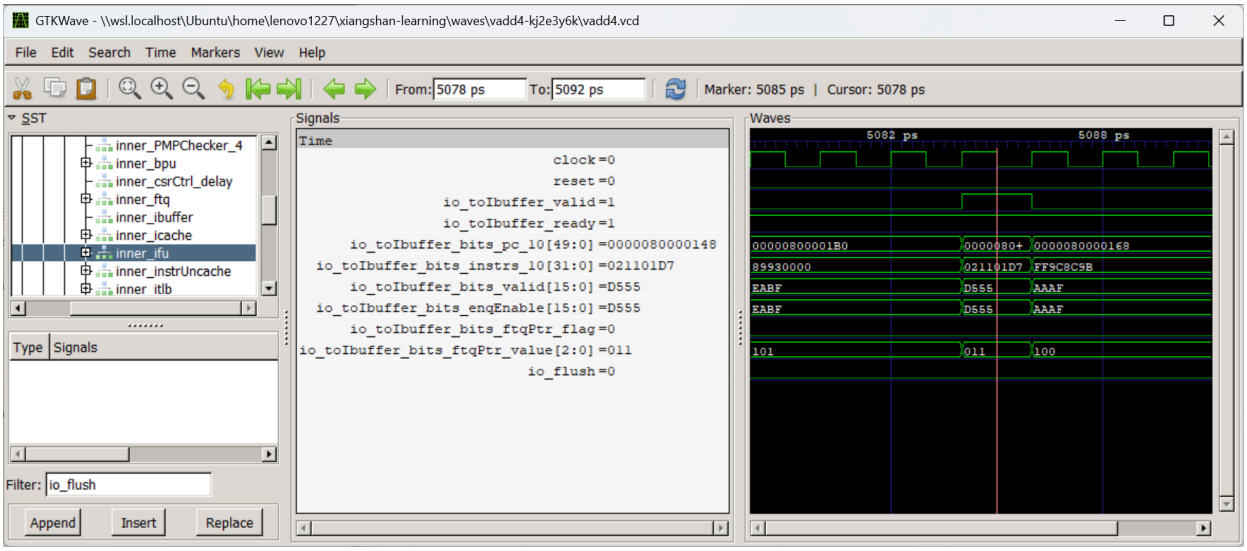


图 3-6　IFU 位置 10 的指令身份、包内掩码和接口握手共同确认接收。

到 5087 ps，目标位于 IBuffer 输出槽 5，FTQ 仍为 `0/3`，`ftqOffset=0xA`。IFU 采用半字位置编号，IBuffer 输出采用指令槽编号，因此从位置 10 变成槽 5，只是编号方式不同，仍是同一条指令。

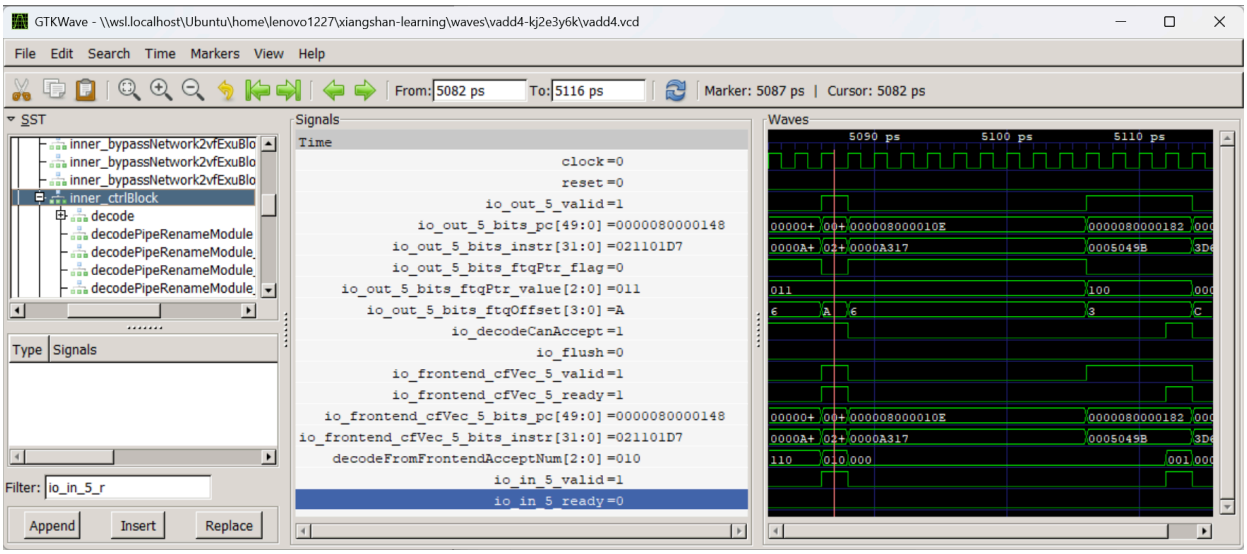


图 3-7　后端入口 `cfVec5` 的 valid/ready 为 1/1，但译码输入 ready 为 0，指令先由入口缓冲保存。

3.3.3 decodeBuf 等待与译码接受

5088 ps，后端入口接收目标指令。当时译码只接收了前两条指令，目标从输出槽 5 转入 decodeBuf 槽 3。前方指令继续被消费后，目标在 5090、5092 ps 依次移到槽 1、槽 0，PC、机器码和 FTQ 标识保持一致。

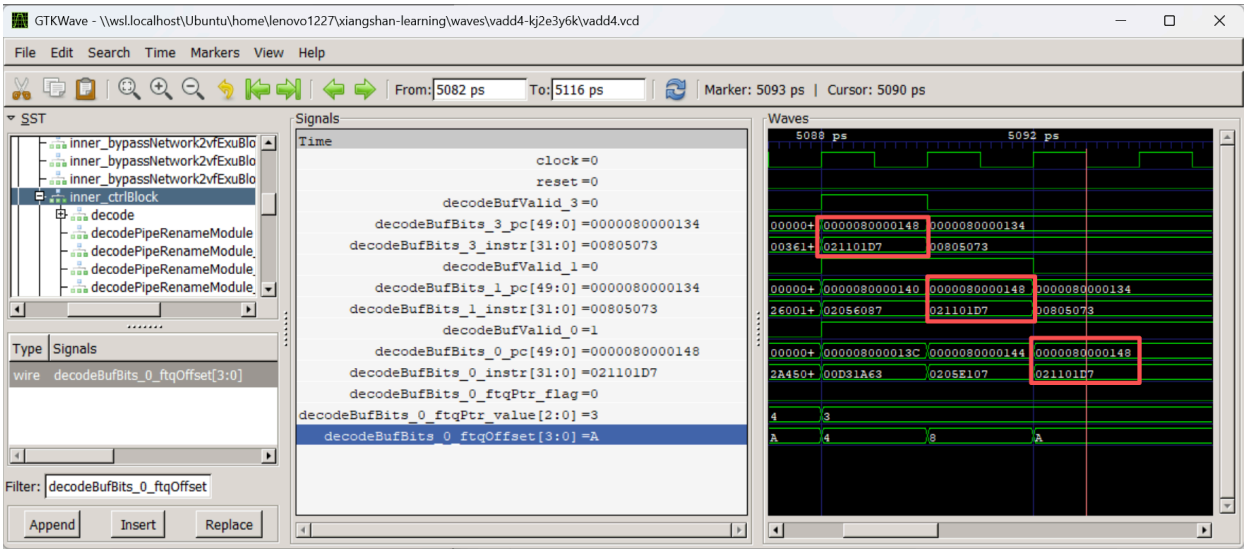


图 3-8　5093 ps 目标位于有效的 decodeBuf 槽 0；这只是缓冲槽位变化，仍是同一条目标指令。

目标到队首后仍需等待译码 ready。5111 ps 的输入 valid/ready 才同时为 1，redirect=0，因此 5112 ps 为真正的译码接受沿。沿后 `decodeBufValid_0` 清零，译码入口转向下一条存储指令 `0x8000014c`。入口 valid 可以继续为 1，因为此时它已经描述下一条指令。

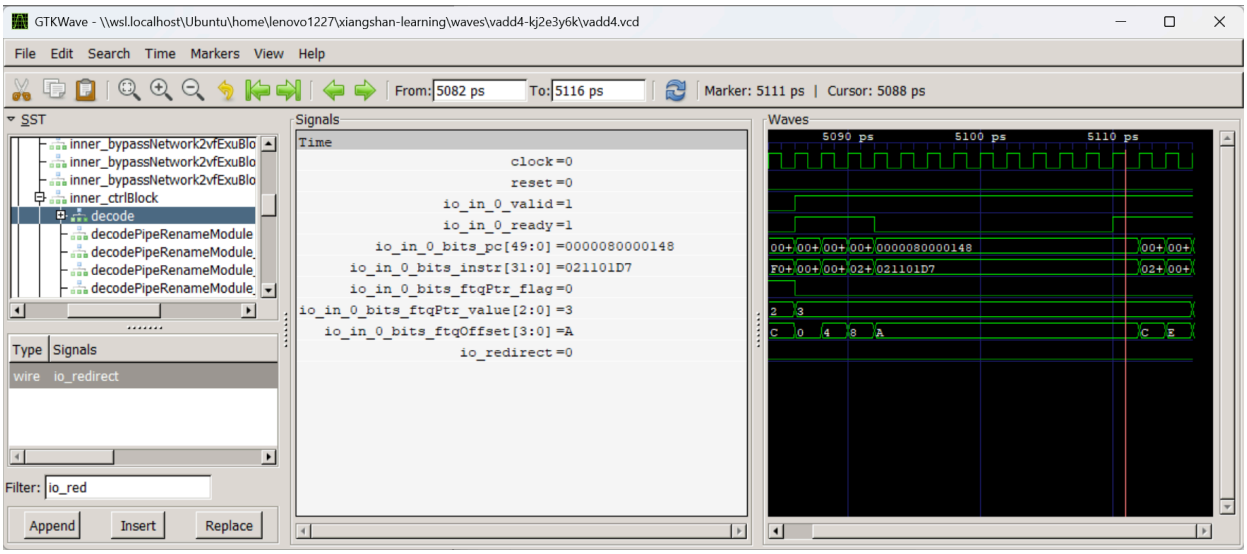


图 3-9　5111 ps 的稳定握手值确定 5112 ps 的译码接受事件。

3.4 寄存器重命名、等待与发射

3.4.1 译码建立运算和源类型

译码将机器码解释为向量加法：逻辑源为 v2、v1，逻辑目的为 v3，`vecWen=1`、`rfWen=0`；`fuType=0x80000`、`fuOpType=0x40` 对应本版本的向量整数加法路径。向量控制 `vsew=2`、`vlmul=0`、`vm=1`，即 e32/m1 无掩码操作。

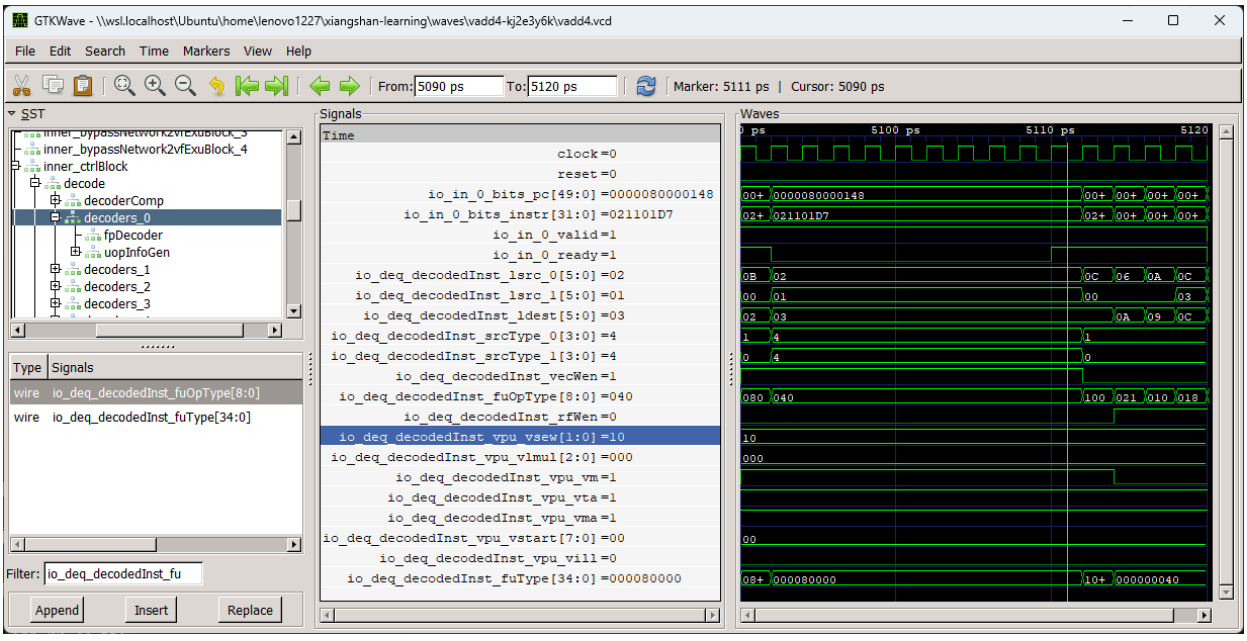


图 3-10　译码把指令编码转换为逻辑源、目的及执行控制。两个源类型均为向量类型。

汇编按 `vd,vs2,vs1` 排列，而本实现的源 0/1 按 RS1/RS2 字段提取，因此该实例的源 0 对应 v2，源 1 对应 v1。通用 old-vd 字段仍能看到逻辑 v3，但本例 `isDependOldVd=0`，旧目的不参与加法。

3.4.2 重命名分配物理版本

5115 ps，重命名输出把 v2、v1 映射到物理源 50、48，为新的 v3 分配物理目的 51，同时携带 ROB `1/16`。valid/ready 为 1/1，输出在 5116 ps 被接受。波形采用十六进制时，这些编号分别显示为 `0x32`、`0x30`、`0x33`，ROB value 为 `0x10`。

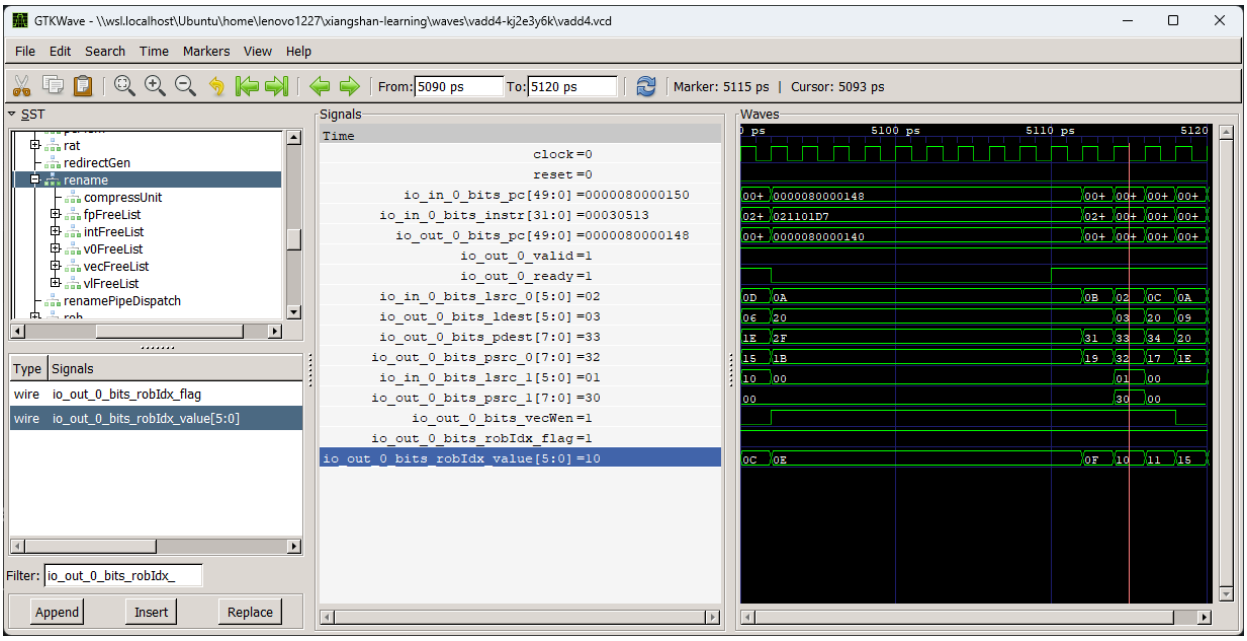


图 3-11　重命名后的物理源、物理目的和 ROB 身份，为后端追踪提供共同标识。

重命名先为本次写入准备新的物理寄存器，已有输入还要等前面的指令算完。5117 ps，派发接口 valid/ready 为 1/1，说明队列能接收 VADD；两个 `srcState` 都为 0，说明两个输入尚未就绪。VADD 在 5118 ps 入队后继续等待。

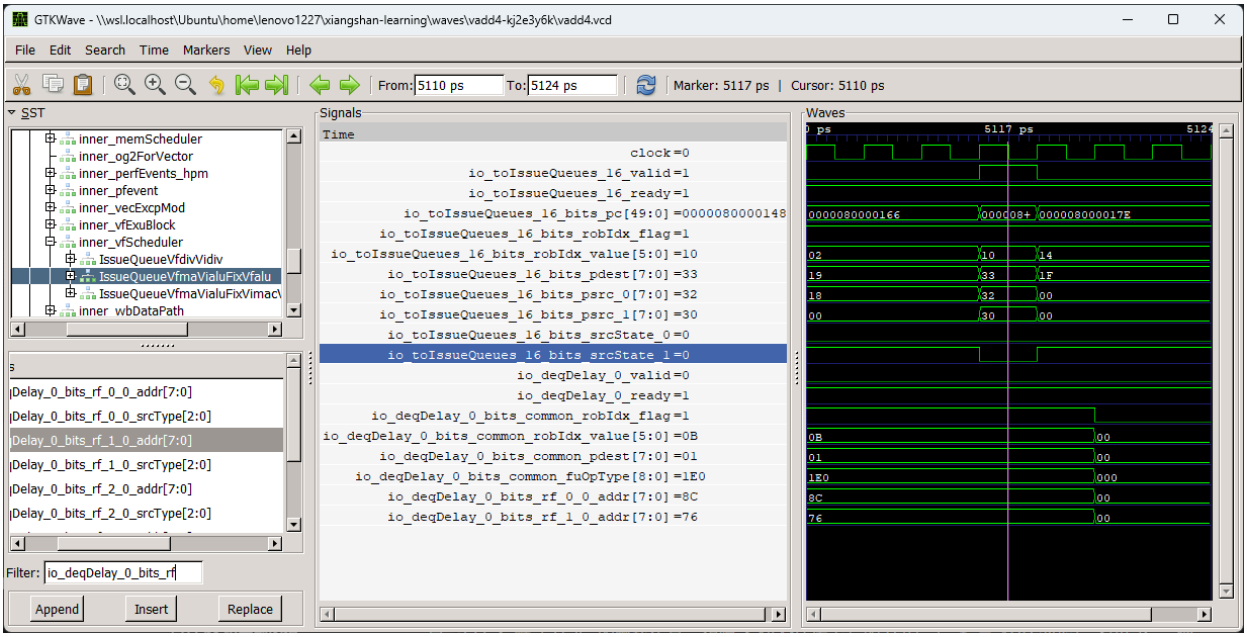


图 3-12　目标进入发射队列时两个源尚未就绪。派发 ready 表示发射队列还能接收新条目；操作数是否就绪由 srcState 另行表示。

3.4.3 写回唤醒与发射

5251 ps 的有效写回标签为 48，使源 1 在 5252 ps 就绪；5257 ps 的标签为 50，使源 0 在 5258 ps 就绪。到 5259 ps，条目的 `canIssue=1`。队列随后选择该条目，5261 ps 的发射接口 valid/ready 为 1/1，携带 ROB `1/16`、pdest 51 和两个物理读地址 50、48，于 5262 ps 接受。

VADD 从入队到发射等了较长时间，主要是在等两个源数据和调度机会。计算本身的用时要从功能单元真正接收操作数时开始计，不能把前面的排队时间都算成加法延迟。

3.5 运算、写回与按序提交

3.5.1 实际操作数与计算完成

5267 ps，父执行单元 EXU2 的入口握手有效，ROB/pdest 与此前相同。两个 128 位源按 32 位分组为：

```
src0 = 00000028_0000001e_00000014_0000000a
src1 = 00000004_00000003_00000002_00000001
VL   = 4
```

右端为元素 0，因此它们分别是 B 和 A。5268 ps 接受输入后，5269 ps 的 `VialuFix` 输出已经携带完整和，输出 valid/ready 为 1/1，5270 ps 完成握手。

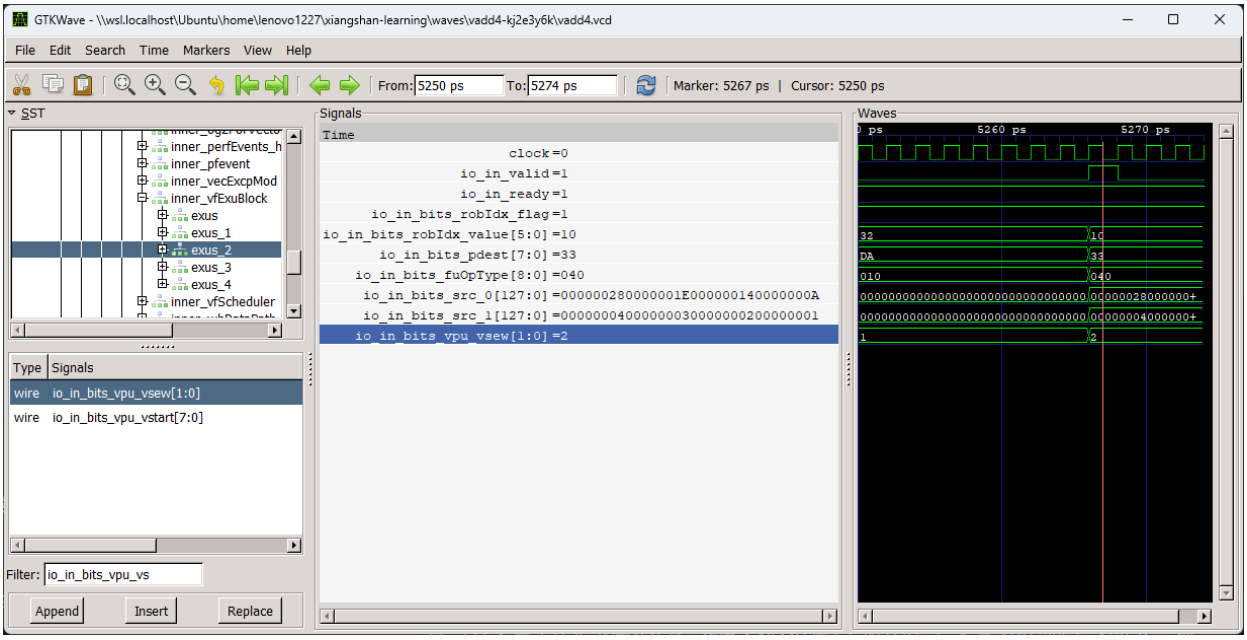


图 3-13　5268 ps 接受的操作数为两组四元素向量，ROB `1/16` 和 pdest 51 与重命名一致。

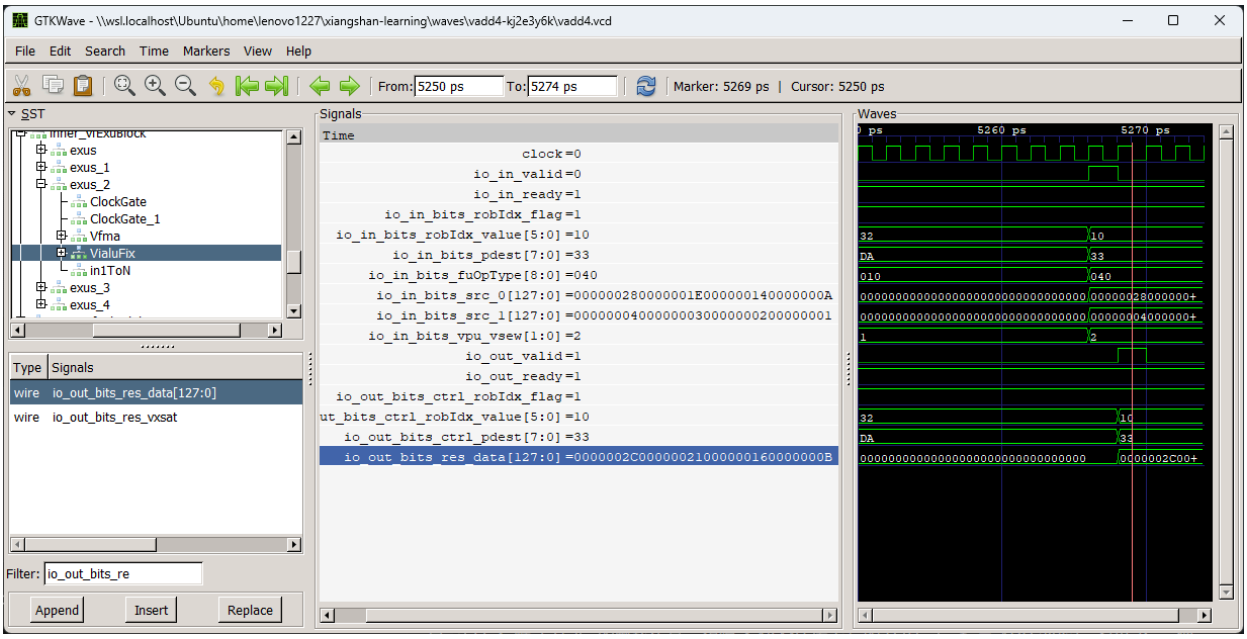


图 3-14　5270 ps 输出的 128 位结果为 `0x0000002c000000210000001600000000b`，对应 `[11,22,33,44]`。

3.5.2 物理写入、退休和映射更新

运算输出经写回数据通路送到向量寄存器堆。5271 ps 写口 `wen=1`、地址为 51，5272 ps 四个 32 位分片的 `mem_51` 分别写入 `0x0b`、`0x16`、`0x21`、`0x2c`。此时结果已经存在于物理寄存器中。

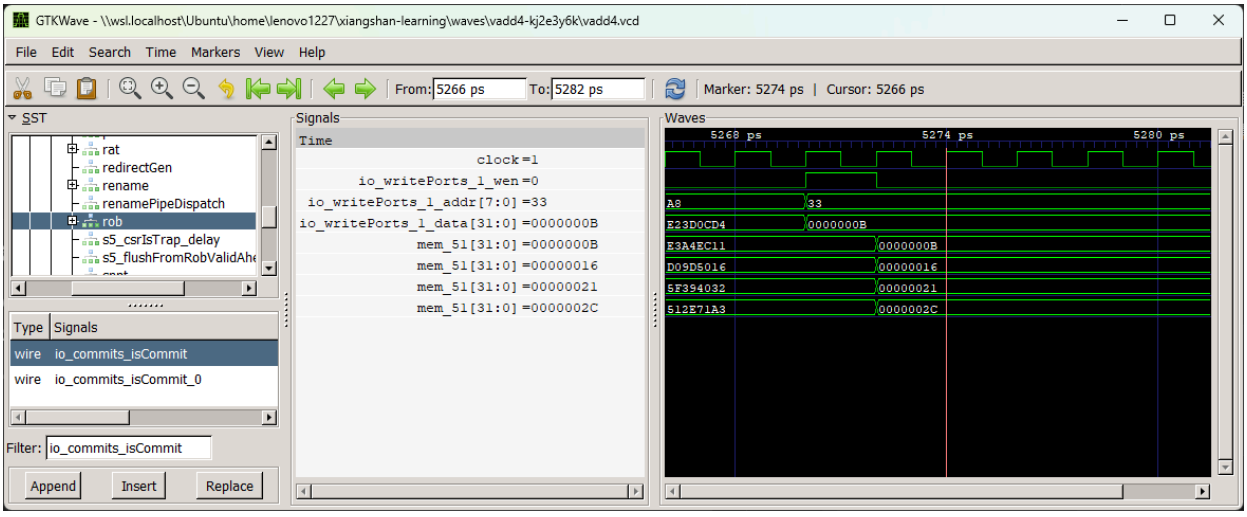


图 3-15　物理写入发生于 5272 ps，图中 Marker=5274 ps 时各分片保持新值。

随后，5273 ps 的 ROB 退休槽 0 满足 `isCommit=commitValid_0=1`，PC、机器码、ROB `1/16` 和 pdest 51 均匹配目标，指令在 5274 ps 按序退休。这一步确认该结果已按程序次序提交。

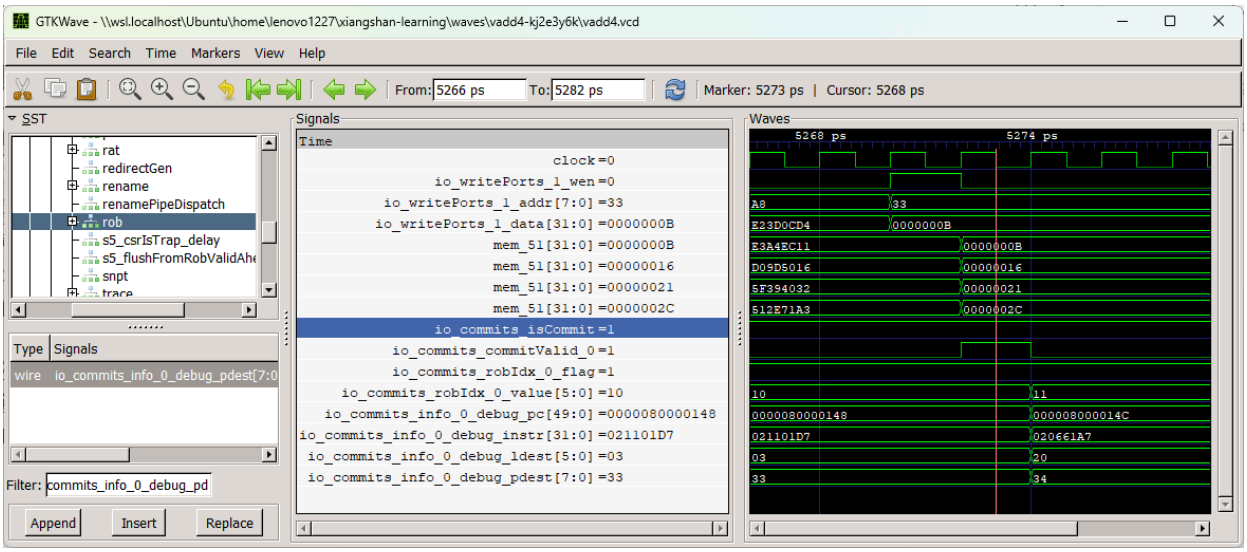


图 3-16　5273 ps 的有效退休槽确认 5274 ps 退休事件。

退休信息再经 RAB 更新向量已提交映射。5277 ps，映射写口准备把逻辑 v3 指向物理 51，`arch_table_3` 尚为旧编号 3；5278 ps 沿后，表项变为 51。

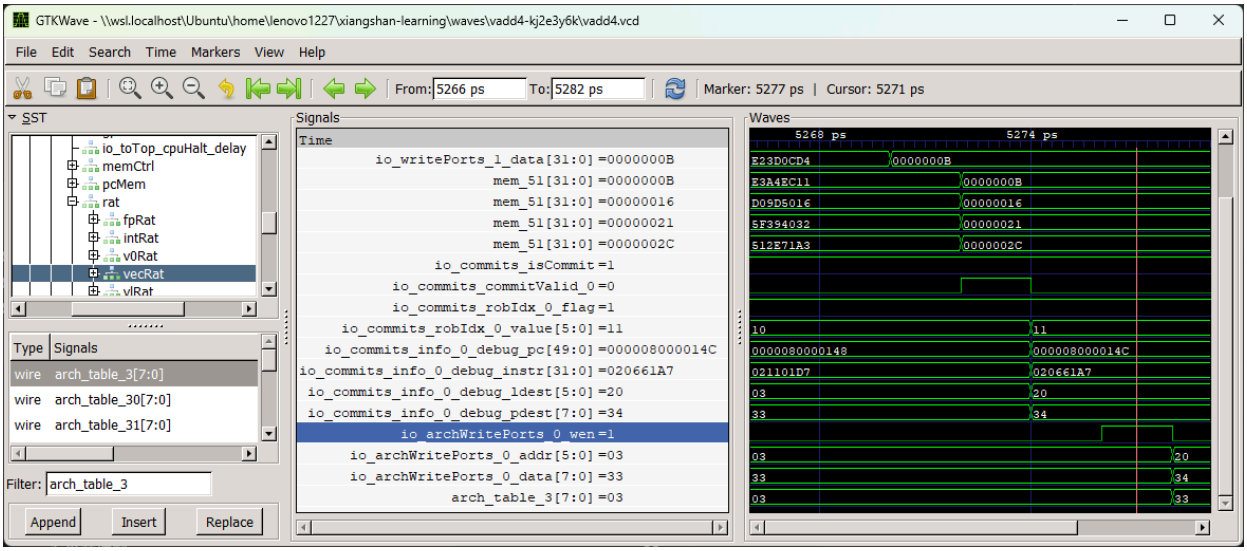


图 3-17　5277 ps 的写口为 addr=3、data=51，已提交表项仍为旧编号 3。

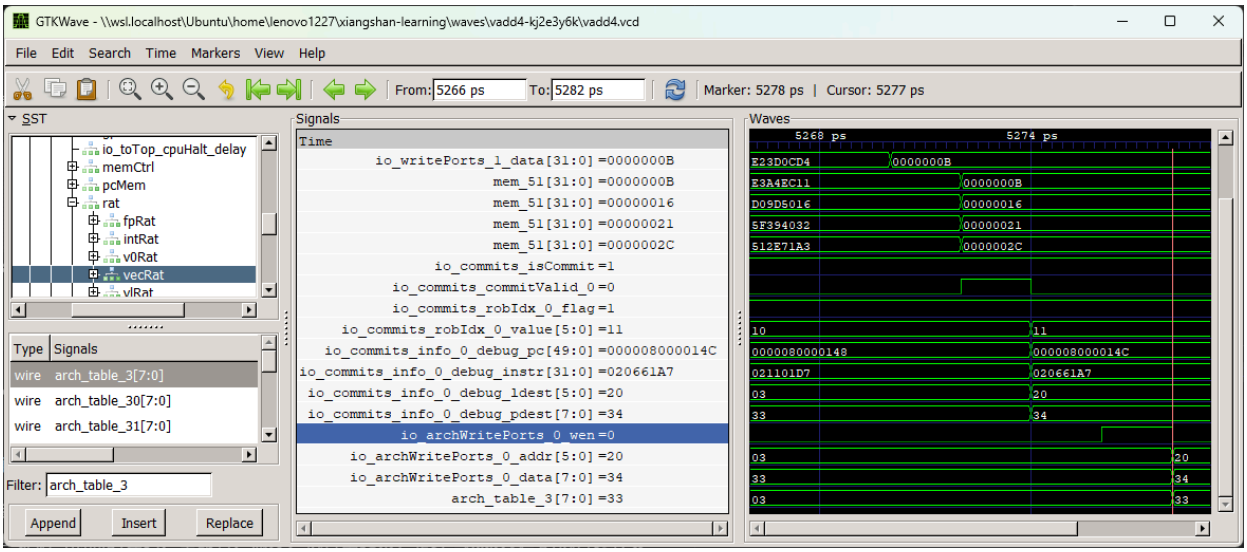


图 3-18　5278 ps 后 v3 的已提交映射指向物理寄存器 51。

这次执行有三个不同的时间点：5272 ps，结果写进物理寄存器；5274 ps，ROB 确认这条指令可以按程序次序退休；5278 ps，已提交映射表记下“v3 对应物理 51”。映射表只保存编号，真正的 128 位数据仍保存在物理寄存器 51 中。前端的 FTQ 和后端的 ROB、物理目的编号，把这三个结果与最初取回的 VADD 连在一起。

本章覆盖 e32/m1/VL4 实例从 FTQ 请求、源依赖等待到提交更新的过程。缓存缺失处理、BPU 内部预测及其他向量配置属于后续扩展观察内容。复现材料和信号索引见阶段一附录。

附录 A 环境与波形索引

本附录列出第一阶段的源码版本、二进制校验值及完整运行记录。主要构建、运行命令和波形分析见第 2、3 章，下列身份信息对应保存的验收记录。

A.1 源码与二进制身份

表 A-1　源码版本。

对象	提交
XiangShan	16ae9ddcda54fc9a2fddffad73174cf793ac7814
nexus-am	086a238062fdfb3281b745a537b361983d0154b5
difftest	62e4540c03c03aa947bdb67f4b6bf66f1a29c5d0
ready-to-run	ed8c30e1ec8a9c9aa6b116ce54281235d00273fa

表 A-2　第一阶段二进制文件 SHA256。

文件	保存的 SHA256
emu	9ee6ce3e8372c6345fc7219563c5ac63c9b3517adb87dc347816d14f228fd7e2
预编译参考库	7af2120acefa83b7299c91b99132f4087dd8ab0f7219461b9aea48ce38891e03
Hello BIN	195dca0c3a748fea70efa84e1d63cd3926098189f61fc6babe80214425b1f479
VADD BIN	0fa5845070b3f954cf375cf571692326971cda6a994d1f20ee65a626d4357283

恢复时按第 2 章准备工具和源码，分别构建 emu 与 AM 应用；按第 3 章导出新 VCD，并保留运行日志与退出状态。以上校验值用于定位历史实验所用的二进制文件；在新环境重新编译后，另行记录文件校验值。

A.2 完整运行和事件记录

内容	工程入口
环境与版本核对	验收记录〔5〕
Hello 完整运行包	源码、镜像、日志与身份〔7〕
VADD 应用构建	编译文件清单〔8〕
VADD 导波形	运行及退出状态〔9〕、波形清单〔10〕
VADD 汇编	源程序〔11〕
前端逐时刻观测	前端记录目录〔12〕
执行阶段观测	操作数与完成事件〔13〕
写回与退休观测	物理写入、退休及映射〔14〕

A.3 波形信号与源码导航

以下层级位于 TOP.SimTop.l_soc.core_with_l2.core 下，源码相对锁定版本的 src/main/scala/xiangshan/。这些名称用于在本次 VCD 中定位信号；事件是否成立，仍以正文给出的 valid/ready 等判定条件为准。

表 A-3　前端定位。

阶段	波形层级及主要信号	源码
FTQ	frontend.inner_ftq：io_toIfu_req_valid/ready、startAddr、ftqIdx	frontend/FrontendBundle.scala、Frontend.scala
ICache	frontend.inner_icache：io_fetch_resp_valid、vaddr、doubleline、data	frontend/icache/ICacheMainPipe.scala
IFU	frontend.inner_ifu：f2 FTQ、f2_fire、f2_cut_ptr_10/11、toIbuffer 位置10	frontend/IFU.scala
IBuffer	frontend.inner_ibuffer：io_out_5_*、decodeCanAccept、flush	frontend/IBuffer.scala
后端入口	backend.inner_ctrlBlock：cfVec5、decodeBuf 槽3/1/0	backend/CtrlBlock.scala
译码入口	backend.inner_ctrlBlock.decode：in0 valid/ready、PC、指令字	backend/decode/DecodeUnit.scala

表 A-4　后端定位，层级在 backend 下。

阶段	波形层级及主要信号	源码
重命名	inner_ctrlBlock.rename：out0 逻辑/物理编号、ROB	backend/rename/Rename.scala
派发	inner_ctrlBlock.dispatch：io_toIssueQueues_16_*	后端派发逻辑
唤醒发射	inner_vfScheduler.IssueQueueVfmaVialuFixVfalu：wakeUpFromWB4、srcStatus、deqDelay0	backend/issue/EntryBundles.scala、IssueQueue.scala
执行	inner_vfExuBlock.exus_2、VialuFix：src、res.data	backend/fu/vector/VecPipedFuncUnit.scala、backend/fu/wrapper/VIAluFix.scala
物理写入	inner_dataPath.vfRegFilePart0～Part3：writePorts1、mem51	backend/datapath/DataPath.scala、backend/regfile/Regfile.scala
退休	inner_ctrlBlock.rob：isCommit、commitValid0、身份	backend/rob/Rob.scala
已提交映射	inner_ctrlBlock.rat.vecRat：archWritePorts0、arch_table3	backend/rename/RenameTable.scala

本例的 PC、ROB/pdest 和时标见第 3 章。其他运行需按其自身记录重新核对配置、信号层级与指令标识。

材料完整路径

编号与本阶段正文一致。路径相对于配套 RISCV_vdot 工程根目录；图片随本文档目录提供，完整源码、日志和大型 VCD 由配套工程提供。

编号	材料	项目相对路径
1	部署记录	artifacts/deployment-history-2026-10-19/build-retry.log
2	构建时间记录	artifacts/stage1-environment-review-2026-10-19/history/build-time.log
3	Hello 原始日志	artifacts/runs/S1-01/hello-meijie-20261019-120654-8FtdkQ/run.log
4	退出记录	artifacts/runs/S1-01/hello-meijie-20261019-120654-8FtdkQ/run-result.txt
5	环境核对记录	artifacts/stage1-environment-review-2026-10-19/evidence-review.json
6	原波形指南	exercises/vadd4/WAVE.md
7	源码、镜像、日志与身份	artifacts/runs/S1-01/hello-meijie-20261019-120654-8FtdkQ
8	编译文件清单	artifacts/vadd4-2026-10-17/manifest.json
9	运行及退出状态	artifacts/vadd4-wave-2026-10-17/run-result.json
10	波形清单	artifacts/vadd4-wave-2026-10-17/manifest.json
11	源程序	exercises/vadd4/vadd4.S
12	前端记录目录	artifacts/vadd4-frontend-2026-10-19
13	操作数与完成事件	artifacts/vadd4-execution-2026-10-18/observations.json
14	物理写入、退休及映射	artifacts/vadd4-writeback-2026-10-18/observations.json